UNIVERSITY / FACULTY / DEPARTMENT

Project III – AIT

FINAL REPORT

Implementation, Training and Optimization of LLMs for Automated API Fuzzing on Heterogeneous GPUs

A reproducible pipeline for LLM-assisted deep learning API fuzzing, fine-tuning, and GPU optimization

Instructor: Dr. Kim Đình Thái
Course section: Project III (AIT)
Student: Trần Duy Gia Long (ID: 22071106)
Submission date: 11/06/2026

Hanoi, 2026

Group Members and Contribution Statement

Table 1: Group members and contribution percentages.

No.	Full Name	Student ID	Main Tasks	Contribution
1	Trần Duy Gia Long	22071106	Designed the fuzzing pipeline, built datasets, trained Gemma4/LoRA, optimized TRL–Unsloth–DDP, analyzed results, and wrote the report.	100%

Acknowledgements

I would like to thank the course instructor for the guidance on reproducible experimental work. This guidance helped me organize the project into clear parts: problem definition, data, method, experiment setup, results, discussion, and future work.

This project uses many open-source tools, including PyTorch, TensorFlow, Hugging Face Transformers, TRL, PEFT, Unsloth, Liger Kernel, llama.cpp, and cloud GPU runtimes. These tools were used to build, train, test, and optimize large language models in a mixed GPU environment.

I also acknowledge the public documents and open-source artifacts used to build the fuzzing dataset, including PyTorch, TensorFlow, and Keras API documents, public issue reports, and deep learning fuzzing projects such as DFUZZ, TitanFuzz, FreeFuzz, and NablaFuzz. The implementation in this report is based on notebooks, scripts, logs, technical notes, and project evidence archives created during the project.

Contents

Abstract

This report presents the implementation, training, and optimization of large language models for automated API fuzzing. The main targets are APIs from PyTorch and TensorFlow. Instead of asking a model to write code without control, the project builds a measured pipeline. The pipeline selects target APIs, asks the model to write a small seed program, checks the code with AST rules, runs it in a subprocess with a timeout, generates a fuzzing script, removes fake crashes, and classifies the result. The model part studies Gemma 4 E2B/E4B, official safetensors, quantized GGUF, and LoRA/rsLoRA fine-tuning of `google/gemma-4-E4B-it` on Blackwell GPUs. The experiments use two main environments: Google Colab with an NVIDIA T4 GPU and a cloud runtime with two NVIDIA RTX PRO 6000 Blackwell GPUs. The results show that GGUF Q4 is much faster than official 4-bit inference on T4, with about 6.2x to 11.8x speedup. On Blackwell, Unsloth DDP can run after fixing the Gemma4 proxy and freezing unused vision/audio branches. A key lesson is that adapter correctness must be checked directly: some old adapters were no-op because `lora_B=0`, while newer manual DDP runs passed the B-check. The current best candidate is `gemma4_STD500_3ep_eval035_20260606`, but it still needs wider fuzzing evaluation. A later numerical-reliability stage adds a certified numerical oracle, a seed-sieve pipeline, a common-GPU fuzzer, and small reproducible scripts for linalg, sparse tensor, storage, and compiler-difference surfaces. The project also finds practical bottlenecks such as offline wheel management, FlashAttention limits, a 262K-token vocabulary, padding waste, and repeated model reload time. A resident worker reduces job startup time and makes the pipeline closer to a usable training system.

Keywords: API fuzzing; large language model; Gemma 4; LoRA; Unsloth; TRL; DDP; Blackwell GPU; PyTorch; TensorFlow.

1. Introduction

Deep learning libraries such as PyTorch and TensorFlow are large and complex. They contain many APIs, tensor types, data types, devices, layouts, and backend kernels. Manual testing is not enough for this kind of system. API fuzzing is an important method because it can create many small programs that test unusual inputs, such as empty tensors, zero dimensions, rare dtypes, non-contiguous tensors, NaN/Inf values, autograd cases, CPU/GPU mismatch, and invalid arguments.

Traditional fuzzing also has limits in this area. First, it is hard to generate valid inputs for deep learning APIs because many APIs have shape, dtype, device, and relation constraints. Second, if generated code is too free, it may create fake crashes such as `raise`, `assertFalse`, `sys.exit`, or normal Python errors. These do not show a real backend problem. Therefore, an API fuzzing system needs both valid program generation and strict checking.

Large language models (LLMs) can help because they understand API names, documents, code style, traceback messages, and Python programs. In this project, the LLM is not treated as the final oracle. It is only a generator of candidates. A candidate is useful only after static checks, dynamic execution, and manual or rule-based triage. This view supports defensive security research: the goal is to find possible bugs, reduce manual seed writing, and build a feedback loop between data, model, and harness.

The main contributions of this report are:

(1) Build a DFUZZ-style API fuzzing pipeline: API selection, seed generation, validation, subprocess execution, fuzz script generation, target-call instrumentation, and result classification.

(2) Build and clean SFT data for a fuzzing assistant, including high-quality synthetic data, public issues, fuzzing artifacts, and Vietnamese instruction data.

(3) Fine-tune and optimize `google/gemma-4-E4B-it` with LoRA/rsLoRA on a cloud runtime with two RTX PRO 6000 Blackwell GPUs.

(4) Compare several model formats and environments: official safetensors, bitsandbytes 4-bit, GGUF Q4, clean TRL, Unsloth single GPU, Unsloth DDP, and a resident worker.

(5) Analyze key engineering failures: offline runtime, missing wheels, `huggingface_hub` conflicts, `HybridCache`, Gemma4 proxy issues, unused vision/audio branches, attention head limits, and OOM from a 262K-token vocabulary.

(6) Organize an evidence pack with slides, reports, benchmark JSON files, notebooks, runbooks, HAR-to-markdown files, issue packs, checkpoints, merge scripts, and offline packs.

(7) Summarize inference optimization on Blackwell: vLLM, MTP speculative decoding, NVFP4 Marlin, native FP4 b12x, CUDA 13, and the `sm120f` architecture suffix.

(8) Link the LLM pipeline with an adaptive fuzzing loop and responsible disclosure practice, including Tier A crashes, Tier B contracts, Tier S side effects, small reproducers, and manual triage.

(9) Add the numerical-reliability line: a certified tensor-program oracle, surprise-based search, fault-injection sensitivity testing, and iterative self-training data export for later Gemma4 fine-tuning.

(10) Add a common-GPU fuzzer with reproducible subprocess probes for linalg, sparse tensors, indexing, storage metadata, and eager/compiled comparison.

2. Materials and Methods

2.1. Problem Scope

The task is defined as follows. Given a list of PyTorch or TensorFlow target APIs, the system should generate a minimal Python seed program that calls the target API. If the seed runs, the system then asks the model to generate a short fuzzing script. Each script runs in a separate subprocess with a timeout. It must not use dangerous behavior or fake crash patterns.

The pipeline input includes:

- target API names, for example `torch.fft.irfft`, `torch.linalg.slogdet`, and `torch.nn.functional.grid_sample`;
- prompts for seed generation and fuzz generation;
- model decoding settings;
- safety rules and result classification rules.

The pipeline output includes:

- seed pass rate;
- one-shot seed rate;
- number of crash or runtime-error candidates;
- number of shallow, setup, invalid, and timeout cases;
- JSON logs that can be checked later.

2.2. API Fuzzing Pipeline

The pipeline is designed to avoid weak manual judgement. Each result must pass static checking and dynamic execution.

Algorithm 1 LLM-assisted API fuzzing pipeline

Select a balanced API set from groups such as `torch`, `torch.Tensor`, `torch.fft`, `torch.linalg`, `torch.nn.functional`, and `torch.special`.
for each target API **do**
  Ask the LLM to write a minimal Python seed program.
  Parse the code with AST checks for syntax, imports, target call, and blocked behavior.
  Run the seed in a subprocess with a timeout.
  if the seed runs **then**
    Ask the LLM to write a fuzzing script from the valid seed.
    Use AST checks to block `raise`, `assertFalse`, `sys.exit`, file/network I/O, `ctypes`, and nested subprocess calls.
    Run the fuzzing script and classify the result as safe, shallow, setup failure, runtime crash, segfault, OOM, or invalid code.
  else
    Log the seed failure reason and skip the fuzzing phase.
  end if
end for
Export JSON reports and summary tables.

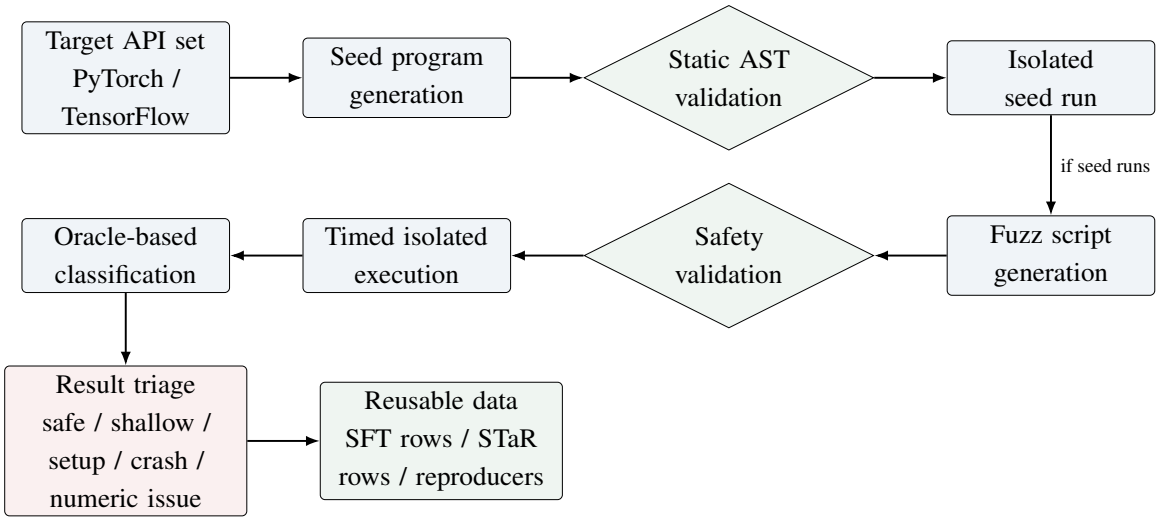


Figure 1: Workflow of the API fuzzing pipeline. The LLM writes seed and fuzz scripts. The harness checks AST rules, subprocess behavior, and oracle results. Useful artifacts are later used as feedback data for Gemma4 fine-tuning.

2.3. Initial Seed Corpus and DFUZZ-style Mutator

Before the Gemma4 benchmarks, the project built a DFUZZ-style seed corpus. The handoff notes show that the seed set grew from 5 initial seeds to 15 Python 3.10–3.13 seeds, then to 25 PyTorch/TensorFlow seeds, then to 30 CVE-surface seeds, and finally to `fuzz_seeds.jsonl` with 35 seeds. This final set contains 22 PyTorch seeds and 13 TensorFlow seeds.

The seeds are not only simple API examples. They cover hard language and library patterns, such as `itertools.batched`, PEP 695 type alias, `sys.monitoring`, `asyncio.TaskGroup`, PEP 688 buffer and `torch.frombuffer`, nested f-strings, `torch.fx`, `typing.override`, `math.sumprod`, `tf.function`, `Unpack`, sparse TensorFlow, ragged tensors, quantization gradients, jagged SDPA, and lifetime/buffer surfaces. The scripts `mutator_prompts.py` and `run_mutator.py` can expand the 35 seeds into about 700 candidate programs if each seed creates 20 variants.

This phase shaped two key choices. First, prompts must be linked to real API invariants. Second, generated code must be checked by a controlled runner instead of being trusted directly.

2.4. Adaptive Fuzzing Loop

The project also uses an adaptive fuzzing loop for deeper PyTorch and TensorFlow surfaces. The rule is written in `FUZZING_LOOP_SOP.md`: one round should be one notebook cell. After each round, the real stdout and stderr must be read before the next round is written. If a surface fails because the environment lacks a library or network, it is dropped. If a surface shows a useful signal, it is expanded with 5–10 orthogonal variants.

For more serious bug hunting, the workflow does not start with free LLM guessing. Before each surface group, the tester reads source code or documents for about 1–2 hours to find API invariants, dispatch paths, rare dtype/layout cases, and edge conditions. Probes run in subprocesses with about 20 seconds timeout, work in `/tmp`, do not use the system shell, and do not call real network services.

Probe results are grouped into three main tiers:

- **Tier A crash**: SIGSEGV, SIGBUS, SIGABRT, SIGFPE, internal assert, or process crash;
- **Tier B contract**: wrong validation, raw errors, unexpected path behavior, or accepted invalid arguments;
- **Tier S side effect**: side effect or silent corruption that needs deeper triage.

To keep the work defensive, the probe code uses neutral names such as `RESEARCH_SIDE_EFFECT`, `contractviolation`, and `inputvalidationgap`. It avoids names such as `evil` or `exploit`. All probes run in a local sandbox, and serious results are prepared for upstream responsible disclosure.

2.5. Certified Numerical Oracle

The newer numerical-reliability line extends the project from crash-only API fuzzing to numerical reliability testing. In this setting, a test case is a small tensor program, not only one API call. A program can contain operations such as `matmul`, `sum`, `softmax`, `logsumexp`, `cumsum`, `exp`, `relu`, and shape-changing operations. The same program is run through several paths, for example eager, compiled, CPU, CUDA, layout changes, split-k reassociation, or mixed precision.

The main idea is to separate measurement from judgement. The measurement step records raw results, exceptions, and numerical differences. The judgement step uses a shared oracle. For floating-point results, the oracle compares the observed value with a high-precision reference and a certified error envelope based on standard rounding-error bounds. If the observed error is outside the envelope, the result is marked `CERTIFIED`. If it is near the boundary, it is marked `GRAY`. Otherwise, it is marked `SILENT`. This is more careful than a single global tolerance such as 10^{-4}, because different programs have different safe error limits.

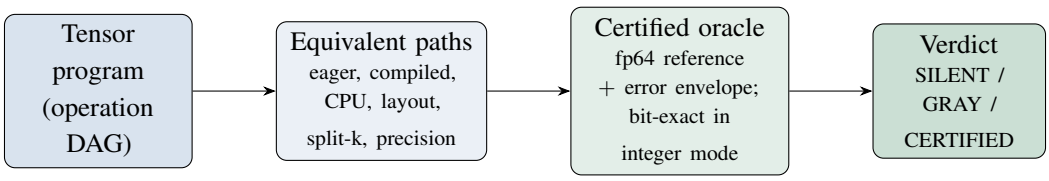


Figure 2: The certified numerical oracle separates measurement from judgement. The same tensor program is executed along several semantics-preserving paths; each result is compared with a high-precision reference and a certified error envelope, or checked bit-for-bit in the integer mode, before a verdict is assigned.

The program prover checks a whole tensor-program DAG instead of checking only one operation at a time. It also exports a conformance certificate, for example `calm_v4_certificate.json`. A later variant keeps the same verdict rules but changes the search fitness. The new search score is

$$\text{surprise} = \text{cert\_ratio} \times \sqrt{k_{\text{eff}}},$$

where k_{eff} is the largest effective accumulation length in the program. This avoids spending too much time on small $k = 2$ cases that are close to the theoretical boundary but are not likely to be backend bugs. This variant also raises the Torch Dynamo recompile/cache limit and exports iterative self-training rows in `calm_star_round1.jsonl`.

A fault-injection stage is used as a sensitivity test for the oracle. Instead of waiting for rare real bugs, it injects controlled defects into `matmul`, such as hidden precision lowering, fp16 substitution, and systematic rounding bias. A good oracle should catch real defect classes, but remain silent on clean controls. This gives a stronger evaluation than only reporting whether the current stable PyTorch release has a new bug.

The workspace also contains a special-value (edge) module. It targets special values such as NaN, $\pm$Inf, signed zero, denormals, and overflow boundaries. At the time of this report, it is

best described as a ready module for the next run, while the program prover and the fault-injection stage already have exported certificates and reports.

The project also includes a hardware-specialized build of the program prover, `calm_axiompp_v5_t4_turbo.py`, specialized for the Tesla T4. Its motivation is that the Tesla T4 (compute capability sm_75) has no TF32 unit, so the precision-substitution class that dominates findings on Ampere cannot appear on this device. The productive surface on the T4 is instead the compiled-fusion path produced by the Inductor backend. A first real-T4 run of the unmodified prover reported no certified cases, in part because the compiled execution path quietly fell back to eager once the Torch Dynamo recompile limit was reached, so the Inductor surface was barely exercised. This specialized build keeps the prover's oracle, the start-up self-test, and the certificate format unchanged, and only widens the compiled execution surface, so it cannot create new false alarms. It makes three changes. First, it raises the Dynamo recompile and cache limits so that the compiled path stays compiled across many program structures instead of silently reverting to eager. Second, it adds a static-shape compiled path next to the dynamic one, because static and dynamic code generation are different Inductor paths with different failure modes. Third, it adds the `amin` and `amink` reductions, which let a single program place several different reductions over the same tensor; this is the pattern behind a reported Inductor partial-reduction reuse defect. Because such a program can be evaluated in the integer "island" mode, where every floating-point operation is exact, the compiled output is checked bit-for-bit against the exact integer result, so a fusion error of this kind is caught with a proof rather than with a noisy finiteness heuristic. The start-up self-test re-validates the error bound of the two new reductions automatically, so an unsound bound would stop the run before any fuzzing begins.

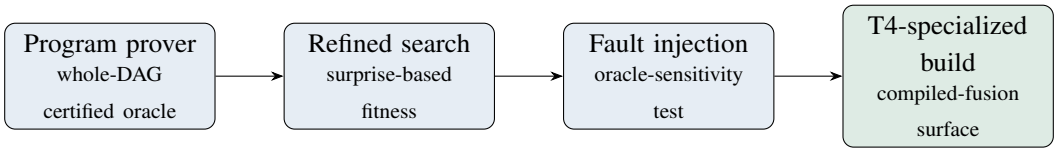


Figure 3: Stages of the numerical-reliability line. The whole-program prover provides the certified oracle; later stages refine the search, measure oracle sensitivity by fault injection, and specialize the compiled-fusion execution surface for the Tesla T4.

2.6. Dataset and Materials

Several data sources were used during the project. Table 2 summarizes the main ones.

Table 2: Main data sources used in the project.

Data source	Scale	Role
GEMMA4_E4B_SMART_SFT_JSONL	25,764 rows	Main SFT dataset. It uses split_group_key, has quality_score, and supports grounded API extraction, Vietnamese fuzzing summary, and safe fuzz target planning.
FUZZ_IDENTITY_INTERNAL_JSONL	9,078 rows	Persona and identity data. It must be used carefully because train/eval leakage is possible.
FUZZ_PARTA / B / B2 / D / E, CORPUS, REFS	many tables	Raw corpus, bugs, traces, and hold-out data. These were filtered into SMART_SFT.
Quality pack 2026-06-06	about 5K train / 81 eval	New data from PyTorch/TensorFlow/Keras issues and public fuzzing artifacts. After filtering/tokenization, 4,979 train rows remained valid in the smoke run.

The SMART_SFT token length distribution is long. The median is about 2,785 tokens, the mean is 2,517, p90 is about 4,399, and p95 is about 4,870. About 58% of samples are longer than 2,048 tokens. This affects max_length, batch size, and padding strategy.

The logs in BAO_CAO_GEMMA4_FUZZING.md show that SMART_SFT was filtered by quality, not only by size. Its minimum quality_score is 70, its average is about 89, and about 64% of rows have direct grounding in APIs or frameworks. The later fast10 run used 3,061 train rows, including 3,000 high-quality rows and 61 controlled identity rows, plus 400 eval rows. It also removed 160 identity-leakage prompts from train/eval and marked 146 duplicate DPO pairs. One important data lesson is that FUZZ_PARTC_VI_PROMPTS_JSONL should not be used directly because about 80% of its outputs are empty.

2.7. Data Preprocessing

The preprocessing stage has three main goals: keep runnable examples, remove unsafe code patterns, and prepare clean prompt/completion training rows. For generated programs, the harness uses AST checks before execution. It checks syntax, imports, target calls, and blocked behaviors. For training data, the project uses a completion-only format. The system and user part becomes the prompt, while the assistant answer becomes the completion. The prompt tokens are masked with -100, so the model learns mainly from the assistant answer.

This choice is important because full-text training can waste most of the loss on system/user text. In one check, about 91.2% of tokens came from system/user/prompt text, while

only about 8.8% came from the assistant answer. After conversion to prompt/completion, the real learned-token ratio in batches was about 14–43%.

2.8. June 11 Seed Sieve and Final SFT Pack

The June 11 workspace implements a systematic multi-stage filtering and data curation workflow to refine raw issue-derived records into high-quality training tokens, bypassing naive keyword matching. The script `calm_seed_sieve_cell.py` processes 7,286 input records through several filtering layers:

1. **AST-based Static Cost-Guard**: It parses candidate code blocks into Abstract Syntax Trees (ASTs) to compute static resource budgets. Any reproducer initializing tensors with a static shape exceeding 2^{24} elements (`cost_numel_budget=2**24`) is pruned to prevent out-of-memory errors on a Tesla T4 GPU.

2. **Platform Capability Check**: Code snippets involving multi-GPU configurations (`torch.distributed`, `DistributedDataParallel`) or external network IO requests are excluded to guarantee single-accelerator runnability.

3. **Judicial Axis Classification**: The metadata and source code of each remaining reproducer are mapped against eight distinct mathematical failure axes representing the taxonomical scope of the oracle: `device_divergence`, `compile_drift`, `precision_dtype`, `metamorphic_axiom`, `layout_phase`, `gradient_adjoint`, `silent_correctness`, and `boundary_contract`.

4. **MAP-Elites Diversity Archiving**: Kept records are archived into a MAP-Elites grid defined by the tuple $(\text{Era} \times \text{Framework} \times \text{Dominant Axis} \times \text{Operator Family})$. To ensure balanced representation, each cell is capped at four high-value seeds (`cell_capacity=4`) and a maximum of two seeds per issue (`per_issue_cap=2`) is enforced.

Through this pipeline, the sieve kept 3,941 records after axis scoring, and populated 174 MAP-Elites cells with 514 high-value seeds. It also distilled a global policy prior (`calm_policy_prior.json`) containing empirical operator weights, active axes pressure, and a watchlist of target API structures.

To compile the final SFT dataset, the sister script `calm_gemma4_sft_dataset_cell.py` integrates LSH-based (Locality-Sensitive Hashing) near-duplicate detection. It computes a 64-bit SimHash signature over 4-gram AST node sequences (stripping comments and variable identifiers), filtering out snippets with a Hamming distance ≤ 6. This prevents data leakage across the dataset splits.

To enable steerable LLM-based generation, the inputs are reformatted with a structured instruction header prepended with control directives such as `Focus:[DominantAxis]`. The final SFT directory (`gemma4_sft_final`) contains 2,314 training rows (comprising 2,134 sieved issue rows and 180 synthetic oracle exemplars), 85 validation rows, and 85 test rows. The

splits are partitioned strictly by issue key, ensuring that code samples from the same GitHub issue never cross split boundaries. Additionally, the common-GPU fuzzer outputs overlay SFT logs (`calm_common_gpu_gemma4_sft_all.jsonl`, `calm_common_gpu_gemma4_sft_novel.jsonl`, and `calm_common_gpu_gemma4_seeds.jsonl`) to dynamically expand the SFT training distribution with active feedback.

Subsequently, to expand the SFT training scope and ensure robust learning on actual PyTorch code patterns, a larger sieved dataset named `dl_seed_fast` was compiled from the rescued issue archives. This dataset enforces strict single-accelerator execution safety (filtering out huge tensors, network IO, shell commands, and distributed dependencies), AST-based deduplication, and a maximum cap of 4 seeds per issue. The resulting `dl_seed_fast` corpus comprises a total of 4,092 PyTorch records, partitioned into 3,757 training samples, 176 validation samples, and 159 test samples. The taxonomical axis distribution of this final package consists of 1,747 compile-related methods, 1,733 GPU execution snippets, 976 precision/dtype checks, 735 autograd nodes, 734 tensor layout variations, and 159 sparse operator rows, with a median sample length of 597 characters. This cleaned and sieved corpus served as the primary training material for the final SFT v2 training runs on Blackwell.

2.9. Model Architecture and Fine-tuning Method

The main model is `google/gemma-4-E4B-it`. Gemma4 is multimodal and has text, vision, and audio parts. This report uses text-only training data, so only the language part should be fine-tuned. If LoRA or DDP hooks touch the vision/audio branches, DDP may fail because those parameters do not receive gradients in a text-only forward pass.

The main LoRA setup is:

- target modules: `q_proj`, `k_proj`, `v_proj`, `o_proj`, `gate_proj`, `up_proj`, and `down_proj`;
- ranks: r=32 and r=64;
- alpha: 64 or 128;
- `lora_dropout=0.0` or 0.05;
- `use_rslora=True`;
- precision: BF16;
- optimizer: `adamw_torch_fused`;
- `ddp_find_unused_parameters=False` after freezing vision/audio branches.

In the fast10 run, LoRA r=32/alpha=64 created about 69.76 million trainable parameters out of about 8.01 billion total parameters, or about 0.87%. Later quality runs used r=64/alpha=128, giving about 139.5 million trainable parameters.

The training loss is the standard causal language modeling cross-entropy loss, but it is applied only on completion tokens. If y_t is the target token and $p(y_t)$ is the model probability for that token, the loss is:

$$L = -\frac{1}{N} \sum_{t \in C} \log p(y_t),$$

where C is the set of completion-token positions and N is the number of unmasked completion tokens. Prompt positions are ignored with label value `-100`.

2.10. Experimental Setup

The report uses two main hardware environments.

Table 3: Hardware environments and their roles.

Environment	GPU / Runtime	Role
Google Colab	NVIDIA T4, Colab CUDA runtime	Fast benchmarks for official 4-bit and GGUF Q4; light inference and fuzzing tests.
Cloud GPU runtime	2x RTX PRO 6000 Blackwell 95–96 GB, sm_120, `torch2.9.1+cu128`	Gemma4 E4B LoRA/rsLoRA fine-tuning, DDP, Unsloth, TRL, and resident worker experiments.
CPU subprocess	Python subprocess with timeout	Runs seed and fuzz scripts to classify errors without freezing the main kernel.

The cloud GPU runtime often has no direct Internet access. Therefore, required wheels must be prepared before the run and stored as internal artifacts. The project uses offline packs that include model snapshots, wheelhouse files, installer scripts, datasets, and training scripts.

Another operational lesson is that GPU identity must be checked directly. A notebook can accidentally run on a smaller GPU instead of the expected Blackwell runtime. The report therefore checks GPU name, GPU count, memory, and process mapping before a benchmark.

2.11. Provenance and Evidence Pack

The project is not a single notebook. It is a chain of experiments. The git repository at `C:/Users/fagon/Loxi/th5` was checked before expanding the report. The current tracked checkpoint is commit `b03042e` with message “Checkpoint current tracked workspace state”. Many large artifacts are untracked, such as archives, notebooks, logs, and evidence folders. These are read and used as evidence, but they are not all committed to git.

The main evidence pack is in `C:/Users/fagon/Loxi/th6/1`. It includes:

- `deck_final`: final 10-page slide deck about DFUZZ, Gemma4, Blackwell, and FP4;

- `desktop_refs`: Gemma4 reports, Blackwell runbook, optimization journey, NVFP4 notes, and diagnostics;
- `th5_30_remaining`: benchmark folder 30, report JSON, Colab scripts, and DFUZZ/Gemma4 result tables;
- `th5_31_remaining`: adapter merge scripts, streaming safetensors merge, CPU fallback, and recovery cells;
- `desktop_project_related`: wider archive with fuzz loop, findings, issue packs, HAR files, fair HF eval, raw output, checkpoints, and older notebooks;
- `har_conversations`: HAR conversations converted to markdown for process reconstruction. They are not public report material because they may contain sensitive metadata.

According to `MOVED_FILES_PROVENANCE.md`, at least 253 files, about 3.23 GB, were moved into the evidence pack from roots such as `th5/30`, `th5/31`, Desktop, and Downloads. This helps trace the source of results because many important logs are outside the main git repository.

The newer numerical-reliability artifacts are under `C:/Users/fagon/Loxi/th6/11`. The most useful organized folder is `SESSION_20260611_CALM_AXIOMpp`. It separates the work into dataset pipeline, training, oracle method, program prover, common-GPU hunt, and common-GPU fuzzer folders. The important files are:

- `gemma4_sft_final/`: final train/validation/test JSONL files and training config;
- `calm_gemma4_train_blackwell.py`: one-file Blackwell training script for Gemma4 fuzzing SFT;
- `calm_axiompp_v4_program_prover.py`, `calm_axiompp_v5_surprise_star.py`, and `calm_axiompp_v6_faultlab.py`: certified oracle, surprise/STaR loop, and oracle sensitivity test;
- `calm_v4_certificate.json`, `calm_v5_certificate.json`, and `calm_v6_faultlab.json`: machine-readable evidence from the prover runs;
- `calm_common_gpu_fuzzer.py`, `calm_common_gpu_fuzzer_summary.md`, and `calm_common_gpu_fuzzer_findings.json`: common-GPU fuzzer and summary;
- `calm_common_gpu_repros/`: minimized subprocess repro scripts;
- `common_gpu_fuzzer_artifacts.zip`: packed repro and report artifact for transfer;
- `SnowflakeStage`: The Snowflake compute stage at `@ML_DB.MODELS.FUZZ_DATASET_STAGE` containing the sieved training datasets (`dl_seed_fast/`), LoRA adapters, and the 22.3 GB merged model shards (`finetuned_gemma4_12b_calm_v2/merged_model/`) to guarantee offline reproducibility;

- `notebooks/`: a collection of 28 Jupyter notebooks representing the offline Colab and Blackwell experiment iterations, renamed to neutral academic identifiers to match the project taxonomy:
  - `gemma4_sft_training_process.md`: progress log documenting SFT dataset compilation, version patches, and multi-GPU training metrics;
  - `gemma4_12b_lora_ddp_training_blackwell.ipynb`: interactive training notebook executing the LoRA DDP run on 2x Blackwell GPUs in Snowflake, saved with its HTML log (`gemma4_12b_lora_ddp_training_blackwell.html`);
  - `gemma4_12b_inference_benchmark_blackwell.html`: benchmark report detailing evaluation of Gemma4 12B token generation throughput on Blackwell GPUs with static KV cache activation;
  - `gemma4_12b_program_prover_fuzzer_t4.ipynb`: T4-optimized program-prover execution log conducting base-vs-fine-tuned differential sampling and execution-gate evaluation on Colab, saved with its HTML log (`gemma4_12b_program_prover_fuzzer_t4.html`);
  - `gemma4_program_prover_fuzzer_v5.ipynb`: program-prover implementation with GPU execution checks, saved with its rendered HTML output (`gemma4_program_prover_fuzzer_v5.html`);
  - `gemma4_program_prover_fuzzer_v4.ipynb`: earlier iteration of the tensor program prover engine;
  - `tensor_program_prover_v4.ipynb`: certified oracle prover with program-level Wilkinson–Higham bounds;
  - `tensor_program_prover_v5_t4.ipynb`: T4-specialized prover build with TF32 precision-class targeting;
  - `pytorch_subprocess_crash_hunter.ipynb`: subprocess-isolated offline crash hunter for PyTorch APIs;
  - `framework_crash_fuzzer_engine.ipynb`: multi-framework crash fuzzer engine supporting PyTorch and TensorFlow through subprocess sandboxing;
  - `common_gpu_fuzzer.ipynb`: common-GPU fuzzer probing linalg, sparse, indexing, and storage-lifecycle surfaces;
  - `gemma4_12b_quantized_seed_generator.ipynb`: 4-bit quantized Gemma4 12B seed generation on Colab T4;
  - `gemma4_12b_streaming_fuzzer.ipynb`: full 12B streaming fuzz pipeline with Google Drive checkpointing for crash recovery;
  - `gemma4_siblings_gguf_t4_bench.ipynb`: evaluation of larger Gemma4 siblings (26B-A4B vs. 31B PyTorch and E2B vs. E4B);
  - `gemma4_26b_31b_gguf_benchmark.ipynb`: fair GGUF benchmark comparing 26B-A4B and 31B PyTorch siblings on Colab T4;

- `gemma4_26b_vs_e4b_comparison.ipynb`: comparison of GGUF 26B Q3_K_M and E4B BF16 precision profiles;
- `gemma4_e2b_mtp_static_cache_bench.ipynb`: static cache and FP16 benchmark of Gemma4 E2B, demonstrating a baseline of ~ 36 tokens/s on Colab T4;
- `gemma4_e2b_t4_inference_bench.ipynb`: comprehensive E2B-it inference benchmark on T4 with batch sweep, MTP assistant, vLLM backend, and continuous batching probes;
- `gemma4_e4b_mtp_assistant_bench.ipynb`: speculative drafting bench using Gemma4 E4B-it and MTP assistants;
- `colab_gguf_fuzz_harness_benchmark.ipynb`: Step-II seed and harness generation bench using GGUF models;
- `numerical_reliability_fuzz_engine_cpu.ipynb` and `numerical_reliability_fuzz_engine_ultimate.ipynb`: standalone CPU and ultimate differential-testing engine runs;
- `numerical_reliability_fuzz_ultimate.ipynb`: full-scale numerical reliability fuzz run with MAP-Elites diversity archive;
- `numerical_reliability_codex_optimized.ipynb`: optimized numerical-reliability codex with oracle integration;
- `numerical_reliability_oracle_v4_executed.ipynb`: certified oracle v4 run with execution outputs preserved;
- `numerical_reliability_oracle_v5_executed.ipynb`: certified oracle v5 run with refined search variant and execution outputs;
- `numerical_reliability_fuzzer_ultimate.ipynb`: fuzzer harness with proof-guided Wilkinson–Higham bounds;
- `github_issue_crawler_2024_2026.ipynb`: GitHub API issue pagination crawler and SFT seed extraction pipeline;
- `github_issue_dataset_builder.ipynb`: standalone 2024–2026 GitHub issue crawler with train/val/test split export for SFT dataset curation.

2.12. Engineering Issues and Fixes

Table 4 summarizes important technical issues and their fixes.

Table 4: Important engineering issues and fixes.

#	Issue	Fix
1	The runtime did not support `gemma4`.	Installed Transformers 5.x offline from wheelhouse while keeping `torch2.9.1+cu128` for Blackwell.
2	Old `peft` conflicted with `HybridCache`.	Used a shim or updated PEFT; tested import in subprocess before training.
3	`torchrun` pointed to the wrong base Python.	Used `sys.executable -m torch.distributed.run`.
4	Offline runtime missed `huggingface_hub>=1.5`.	Downloaded wheels in Colab and installed them before Transformers/Unsloth.
5	Unsloth/Gemma4 proxy caused `Noconfigfilefound`.	Patched `_Gemma4KVSharedSafeProxy.__getattr__` to pass through to the real object.
6	DDP had unused parameters in text-only Gemma4 training.	Froze all trainable parameters in `vision_tower`, `audio_tower`, `embed_vision`, and `embed_audio`.
7	OOM from logits `batchxseqxvocab` with 262K vocab.	Controlled `BATCHxMAX_LEN`, used gradient checkpointing, tried Liger FLCE, and chose safer batch sizes.
8	Packing with SDPA could leak attention across documents.	Used `packing=False` in the clean completion-only route.
9	Every `torchrun` reloaded the 16 GB model and wasted minutes.	Built a resident DDP worker that loads once and receives jobs through a file queue.
10	Runtime could stop because of credit or suspend.	Saved model, dataset, adapter, and logs to artifact storage before shutdown.
11	`Gemma4Processor` treated direct text calls as image input.	Used the inner text tokenizer, for example `tokenizer.tokenizer`.
12	DDP script disabled gradient checkpointing and OOMed.	Re-enabled `use_gradient_checkpointing="unsloth"`.
13	`datasets5.0.0` was not compatible with runtime `pyarrow`.	Used runtime `datasets4.0.0` and added guards for extension-type errors.
14	New runtime had to use the right 2-GPU node type.	Selected 2x RTX PRO 6000 and used short auto-suspend or manual stop.
15	A notebook could fall back to a smaller GPU.	Checked GPU names, count, memory, and process mapping before benchmarking.

#	Issue	Fix
16	Offline full pack missed small dependency wheels.	Installed main packages with `--no-deps` and upgraded only required wheels.
17	Full-text training made the model learn prompt text.	Converted data to prompt/completion and masked the prompt with `-100`.
18	Some fast adapters might be contaminated by packing.	Marked them as diagnostic artifacts, not final quality results.
19	An adapter can be saved but still be no-op if `lora_B=0`.	Checked $\|B\|$ after steps, after save, and after reload; compared base-vs-tuned logits.

2.13. Evaluation Metrics

The report uses two groups of metrics.

Fuzzing metrics:

- seed pass rate;
- one-shot seed rate;
- discovery rate over all APIs or over runnable seeds;
- invalid code rate;
- discoveries per hour and discoveries per 1K tokens.

Training and optimization metrics:

- tokens/s or samples/s;
- peak VRAM;
- train/eval loss;
- number of trainable parameters;
- model load time, job start time, resume behavior, and checkpoint behavior.

3. Results

3.1. Inference Benchmark on Colab T4

Before Blackwell fine-tuning, the project measured Gemma4 E2B/E4B inference speed on a Colab T4. The goal was to choose a practical backend for fuzzing. Table 5 summarizes the result from `gemma4_t4_speed_probe_report.json`.

Table 5: Speed probe on Colab T4 with 288 output tokens.

Backend	Model	tok/s	download_s	load_s	gen_s
Official 4-bit	Gemma4 E2B	7.205	138.636	55.985	39.972
GGUF Q4	Gemma4 E2B	84.747	23.459	12.289	3.398
Official 4-bit	Gemma4 E4B	5.571	323.880	72.369	51.696
GGUF Q4	Gemma4 E4B	34.770	107.646	22.606	8.283

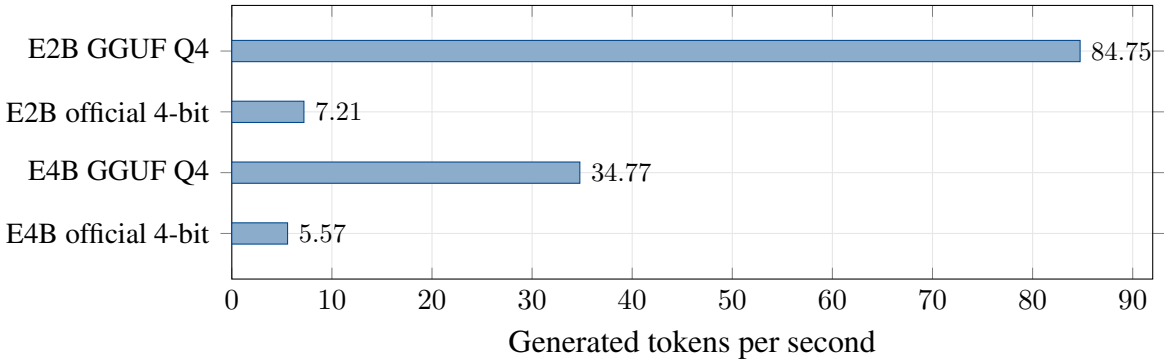


Figure 4: Speed probe on Colab T4. GGUF Q4 is much faster than official 4-bit, so it is useful for wide API fuzzing on T4. Official safetensors are kept for clean baselines and fine-tuning.

GGUF Q4 is much faster than official 4-bit: about 11.8x for E2B and about 6.2x for E4B. Therefore, GGUF Q4 is better for quick fuzzing on T4, while official safetensors are better for standard training and clean comparison.

3.2. Official E2B Two-stage Funnel

Before the GGUF Q4 benchmark, the project tested Gemma4 E2B official 4-bit on 24 PyTorch APIs. The goal was to check whether the seed $\rightarrow$ fuzz pipeline works before backend optimization. Table 6 summarizes the result.

Table 6: Seed/fuzz funnel with Gemma4 E2B official 4-bit.

Metric	Value
APIs tested	24
Runnable seeds	18/24
One-shot runnable seeds	17/24
Runnable fuzzed APIs	18
Runtime crash discoveries	1
Seed pass rate	75.0%
Discovery / all APIs	4.2%
Discovery / runnable APIs	5.6%
Avg seed tokens	207.3
Avg fuzz tokens	235.7

This funnel is weaker than E4B GGUF Q4 in pass rate. Still, it is important because it proves that the two-stage process can run real seeds, filter runtime errors, and create artifacts for later triage.

3.3. GGUF Q4 Fuzzing Benchmark

The benchmark `gemma4_fastest_gguf_fair_fuzzing_report.json` uses 24 APIs from 6 PyTorch groups. E2B and E4B use the same prompts, decoding settings, and timeout. Table 7 gives the main result.

Table 7: Fair fuzzing result between Gemma4 E2B and E4B GGUF Q4.

Model	API	Seed pass	Seed pass rate	Discoveries	Disc./hour
Gemma4 E2B GGUF Q4	24	19	79.17%	1	10.48
Gemma4 E4B GGUF Q4	24	23	95.83%	5	24.01

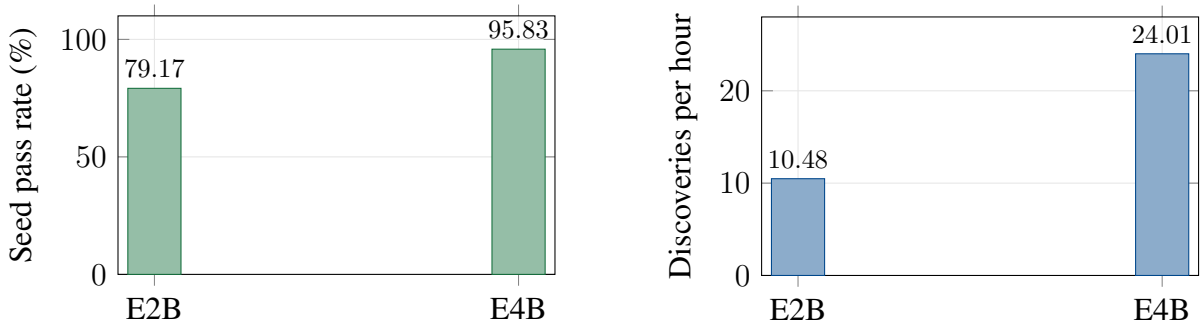


Figure 5: Quantitative comparison of the GGUF Q4 fuzzing benchmark. E4B gives a higher seed pass rate and more runtime-crash candidates than E2B under the same setting.

E4B is better than E2B in both seed pass rate and runtime-crash candidate count. However, every discovery still needs manual checking. Some candidates may be shallow RuntimeErrors, invalid inputs, or setup failures.

3.4. Secondary Archive Benchmark

The folder `desktop_project_related` also contains wider GGUF benchmarks with Gemma4 26B/A4B. These are useful as supporting evidence, but they are not the main result because the label `runtime_crash` mixes shallow errors, RuntimeErrors, and input setup problems.

Table 8: Secondary GGUF 26B/A4B benchmarks in the archive. These are supporting results only.

Model	Attempts	One-shot pass	Feedback pass	Runtime-crash label
`unsloth / gemma-4-26B-A4B-it-GGUF` Q3_K_M	26	19	5	24
`trdgl / gemma4-26b-a4b-qa-GGUF` Q3_K_M	25	9	5	14

The main lesson is not which model has the most raw crashes. The main lesson is that the harness must triage results after generation. Without this step, shallow errors can create a false view of bug-finding power.

3.5. Adaptive Fuzz Loop and Security Findings

The longer adaptive fuzz loop ran on Blackwell with PyTorch 2.9.1+cu128 and TensorFlow 2.21.0. According to `findings_full_summary.md`, after about 36 hours of adaptive fuzzing, the system found 12 initial candidates, retracted 1 after verification, and kept 11 confirmed bug classes for the project research archive. It also created 16 advisory-grade minimal reproducers. These did not overlap with 40 checked references, including recent CVE/NVD/VulDB items and recent GitHub issues in the checked set.

Table 9: Summary of finding classes from adaptive fuzzing.

Group	Research level	Example surface
Data-path memory corruption	Critical/High candidate	`torch.linalg.eigvals`, `torch.linalg.lstsq`, sparse CSR/CSC/COO validation.
CUDA / linear algebra DoS	High candidate	CUDA permanent context kill, `eigvalsh/eigh` with non-Hermitian input.
Silent correctness / training failure	Medium/High candidate	`torch.export` silent zero substitute, dtype promotion precision loss, silent training failure.
Contract / input validation gap	Low–Medium candidate	`searchsorted/bucketize`, `group_norm(num_groups=0)`, negative eps, path normalization.
TensorFlow-specific crash	High DoS candidate	`tf.linalg.cholesky` with non-positive-definite input.

These findings should be described as defensive research results. They need responsible disclosure, minimization, retesting on the newest versions, and upstream feedback before any public CVE-level claim.

3.6. Triaged Compiler, Export, and Validation Findings

The handoff notes and issue drafts also record several reduced cases. Table 10 presents them as technical triage results, not as confirmed public vulnerabilities.

Table 10: Some specific findings triaged from the fuzzing loop.

Surface	Observed behavior	Technical meaning
`nn.GroupNorm(num_groups=0)`	Throws `ZeroDivisionError` instead of validation `ValueError`; reproduced on several PyTorch versions.	Validation gap in constructor/module wrapper. It is a shallow crash but still a contract problem.
`LazyBatchNorm2d` with `complex64`	CPU returns `NotImplementedError`; CUDA returns dtype mismatch `RuntimeError`.	CPU/CUDA behavior is not consistent for complex dtype.

Surface	Observed behavior	Technical meaning
`BatchNorm1d(0)`	Module wrapper rejects zero channel, but functional API can accept a similar case.	Module API and functional API do not share the same invariant.
`torch.compile` with wrong arguments	Eager may throw `IndexError`, while compiled path changes it to `BackendCompilerFailed` or `TorchRuntimeError`.	Compiler changes the exception contract. Eager/compiled comparison is useful.
AOTI/export reductions	Export interpreter matches eager, but AOTI binary has numeric drift for reductions such as `sum`, `mean`, `logsumexp`, and `softmax`.	Correctness drift, not memory corruption. It needs tolerance and backend checks.
`torch.export` with empty parameter storage	Eager rejects a parameter whose storage was resized to zero, but `torch.export.export()` accepts it and later fails.	Export should validate storage/shape invariants earlier.
Wrong sparse COO/CSR metadata	Some sparse invariant violations can lead to SIGSEGV or CUDA device-side assert when checks are disabled.	Sparse invariants are very sensitive. The root cause must be separated from FFT or CUDA reporting noise.

Among these cases, AOTI reduction drift is especially useful for this project because it is a newer surface after the original DFUZZ paper. It also shows that the harness should not only ask for crashes. It should also ask for mismatched exception classes, eager/compiled mismatch, module/function mismatch, sparse invariant problems, storage metadata problems, and numeric drift.

3.7. June 11 SFT Data Pack and Seed Sieve Results

The newer data pass changes the project from an ad-hoc set of examples into a smaller but better controlled SFT package. Table 11 shows the final split.

Table 11: June 11 final SFT package for Gemma4 fuzzing.

Artifact	Rows	Notes
`train.jsonl`	2,314	2,134 public issue rows and 180 synthetic oracle rows.
`val.jsonl`	85	Small validation split for fast checks.
`test.jsonl`	85	Held-out split for smoke and prompt evaluation.
`calm_seed_archive.jsonl`	514	MAP-Elites seed archive from 7,286 raw records after scoring and filtering.
`calm_policy_prior.json`	1 file	Policy-prior data for later generator or Gemma-guided search.

The seed sieve is useful because it does not only keep easy examples. It keeps examples across behavior axes: compile drift, precision/dtype, silent correctness, device divergence, layout phase, metamorphic axioms, boundary contracts, and gradient/adjoint checks. This makes the training data closer to the real task of scientific-software QA.

3.8. Program Prover and Fault-Injection Results

The program-prover runs give a stricter view of correctness than normal fuzzing. A short run in the current workspace checked 5,431 cases and exported `calm_v4_certificate.json`. It reported 5,430 `SILENT` cases and 1 `CERTIFIED` case. A short run of the refined search variant checked 630 cases and also exported a certificate with 629 `SILENT` cases and 1 `CERTIFIED` case. These small runs are not meant to prove that a library is bug-free. Their value is that every reported violation is tied to a proof-style bound and a saved certificate.

The fault-injection test is the stronger oracle-sensitivity check. It injects known defect classes into matrix multiplication and asks whether the prover catches them while staying quiet on clean controls. The result is shown in Table 12.

Table 12: Fault-injection sensitivity result on the local CUDA setup.

Metric	Value
Injected substitution cases	30
`CERTIFIED` substitution cases	29
`GRAY` substitution cases	1
Cases caught by the old global tolerance	28
Cases missed by old tolerance but certified by prover	2
Clean control cases	20
False positives on clean controls	0

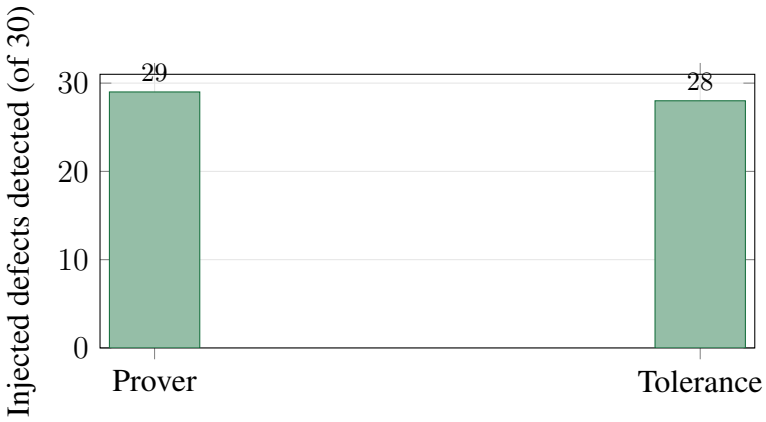


Figure 6: Oracle-sensitivity comparison on 30 injected precision defects. *Prover* is the theorem-guided certified bound; *Tolerance* is a fixed 10^{-4} threshold. The prover certifies 29 defects (one more falls in the gray band) and detects two sub-2×10^{-6} substitutions that the fixed tolerance misses, while keeping zero false positives on 20 clean controls.

This result supports the main oracle claim: a theorem-guided bound can detect hidden precision changes that a fixed tolerance can miss, while it can also stay silent on clean reassociation controls.

3.9. Program Prover Specialization for the Tesla T4

The T4-specialized build was checked first on a local Ampere card and then on the target Tesla T4. On the local card (RTX 3050 Ti, which does have TF32), a 72-second run checked 36,406 cases at about 506 cases per second over 1,276 generated programs. It reported 36,396 `SILENT` cases, 6 `GRAY` cases, and 4 `CERTIFIED` cases. All four certified cases fall in the TF32 precision class, and there were no false positives on the other execution paths. This is the expected behaviour: the oracle still detects the known precision class on hardware that has TF32, while staying silent on the compiled, layout, split-k, and CPU paths. The numbers are summarized in Table 13.

Table 13: Program-prover run with the T4-specialized build on the local Ampere card (RTX 3050 Ti).

Metric	**Value**
Checked cases	36,406
Throughput (cases/s)	506
Generated programs	1,276
`SILENT` cases	36,396
`GRAY` cases	6
`CERTIFIED` cases (all TF32 precision class)	4
False positives on non-precision paths	0
Start-up self-test, including the new reduction bound	PASS

On the target Tesla T4 (sm_75, no TF32), the same notebook runs end to end with the Gemma4 program policy active and the start-up self-test passing. Because the T4 has no TF32 unit, the TF32 precision class cannot appear, so a silent certificate is the expected result on this device rather than a sign of a broken run. The value on the T4 is twofold: a conformance certificate that records every checked (program, path) pair as inside its proven bound, and a compiled-fusion surface that is now actually exercised. During the run the Inductor backend emits an online-softmax reduction-split notice, which confirms that the compiled path is being generated and run rather than reverting to eager. As with the other program-prover runs, these results are not a claim of a new framework defect; on a stable PyTorch release a silent certificate is the correct and useful outcome, and the same notebook can be re-run on a nightly build to raise the chance of a real finding.

3.10. Comparative Program Prover Evaluation on Blackwell

To evaluate the impact of fine-tuning on the program-prover fuzzer framework, a parallel A/B test was conducted on 2× Blackwell GPUs comparing the base model (`google/gemma-4-E4B-it`) against the fine-tuned adapter acting as the program generator. The evaluation measured fuzzing throughput, unique generated programs, coverage cell expansion, and total oracle-tripped findings. Table 14 details the results of this parallel run.

Table 14: Parallel fuzzer A/B test comparison: Base vs. Fine-tuned model (2x Blackwell GPUs).

Evaluation Metric	**Base Model**	**Fine-tuned Model**
Total Checked Cases	**95,977 (400/s)**	93,173 (388/s)
Unique Generated Programs	**3,076**	3,013
Coverage Cell Expansion	**5,419**	4,661
Total Findings (CERT/EXACT)	**3,812**	3,796

The A/B test reveals a near-parity performance, with the base model achieving slightly higher code diversity and coverage cell expansion. This indicates that while the fine-tuned model is highly specialized for generating self-checking seeds, the generic base model retains a broader syntactic diversity that benefits pure exploration. Furthermore, the findings for both models were heavily dominated by the `precision_dtype` axis (due to TF32 precision relaxations on Blackwell) rather than new compiler defects, verifying that the fine-tuning process successfully aligned target formatting behaviors without altering the underlying framework conformance profiles.

3.11. Common-GPU Fuzzer Results

The common-GPU fuzzer is a robust test harness designed to bridge the theoretical certified-oracle line with standard deep learning QA practices. Rather than targeting hardware-specific

kernel compiler bugs, it focuses on general deep learning framework surfaces that manifest consistently across GPU families. The fuzzer structures its execution catalog around four specialized probe surfaces:

- **Linalg Domain Extremes**: Runs operators such as `eig`, `eigvals`, `lstsq`, `solve`, `inv`, `det`, `slogdet`, `svdvals`, `pinv`, `matrix_rank`, `cond`, and `cholesky` with input matrices containing boundary anomalies (`singular`, `zero`, `diag_nan`, `offdiag_nan`, `offdiag_inf`).
- **Sparse Tensor Invariant Gaps**: Investigates how COO and CSR format construction behave when checks are bypassed (`check_invariants=False`) and later converted to dense representations (`coo_neg_to_dense`, `coo_oob_to_dense`, `csr_bad_crow_to_dense`, `csr_oob_col_to_dense`).
- **Index-Select and Embedding Bounds**: Stress-tests out-of-bounds indexing in `nn.functional.embedding`, `gather`, and `index_select` to verify if they raise clean Python errors or crash via raw CUDA device-side assertions.
- **Storage Lifecycle Hazards**: Triggers metadata resize hazards by modifying UntypedStorage size to zero (`s.resize_(0)`) while active tensor references perform operations like `clone`, `add`, `repr`, or `sum`.

To filter out trivial duplicates from the fuzzing output, the fuzzer maintains a triage dictionary of known issue hints. These cover reported patterns like `sgelsy` (NaN handling in matrix solver), `sgebal` (eigenvalue scaling aborts), `silent_invalid_sparse` (quiet sparse corruption), and `srcindex<srcselectdimsize` (standard out-of-bounds device asserts).

In the local execution, the fuzzer evaluated 40 child probes in 157 seconds. It flagged 9 actionable hits, representing 7 unique actionable signatures (summarized in Table 15). Conservative CPU/CUDA exact-difference testing found no unexpected value divergence on clean paths.

Table 15: Common-GPU fuzzer summary.

Metric	**Value**
Executed child probes	40
Actionable hits	9
Unique actionable signatures	7
Conservative CPU/CUDA exact divergences	0
Compile-difference verdicts	30 `ok` cases
Gemma compile-diff seed rows	31
Minimized repro scripts in `calm_common_gpu_repros/`	173

The main actionable families are:

- `torch.linalg.eigvals` with off-diagonal NaN/Inf leading to oneMKL `SGEBAL` backend aborts;
- `torch.linalg.lstsq` with NaN in the input matrix leading to a PyTorch internal assert in the `SGELSY` path;
- sparse COO negative indices that can be materialized silently on CPU when invariant checks are disabled, while CUDA may trigger a device-side assert;
- storage resize-to-zero followed by clone or repr, which can lead to hard exits or contract failures;
- compile-difference seed rows that can be rerun on Linux/Blackwell where Triton is available.

The fuzzer also exports small Gemma4 feedback files. The current local files contain 7 all-row SFT examples, 4 novel/unknown examples, and 31 compile-difference method seeds. These rows are small, but they have high value because they are linked to real repro scripts and triage labels.

3.12. Blackwell Training Script for the New SFT Pack

The training logic for the sieved dataset is implemented in `calm_gemma4_train_blackwell.py`, which is engineered to run in offline environments (e.g., Snowflake 2x RTX PRO 6000 Blackwell GPUs) using PyTorch DDP via:

```
torchrun --standalone --nproc_per_node=2 calm_gemma4_train_blackwell.py
```

The script addresses several critical DDP and offline constraints discovered during initial testing:

- **Multimodal Parameter Freezing**: Since `google/gemma-4-E4B-it` features vision and audio modalities, training on text-only inputs causes DDP to crash due to unused non-language parameters. The script explicitly identifies and freezes 296 multimodal parameters (including `vision_tower`, `audio_tower`, `embed_vision`, and `embed_audio`), isolating training to the language backbone.
- **Offline Unsloth Proxy Patching**: Under strict offline environments, Unsloth's HuggingFace API calls fail. The script applies a runtime passthrough monkeypatch to `_Gemma4KVSharedSafeProxy.__getattr__` to bypass metadata resolution bottlenecks.
- **LoraB Liveness Verification**: To prevent the diagnostic failure of no-op adapters where weights converge to $B \approx 0$, a custom `LoraBLiveness` trainer callback tracks the matrix norm. If the cumulative sum $\|B\|_1$ fails to deviate from zero after a warm-up sequence (`loraB_check_step=20`), the script aborts the run to conserve compute resources.

- **Completion-Only Masking**: Prompts are masked using a token label of `-100`, restricting backpropagation only to the generated code blocks. This forces the model to focus learning capacity on the syntactic structure of python reproducers.

Beyond standard perplexity and eval cross-entropy metrics, the script integrates an automated post-training benchmark named `FUZZ_GEN_EVAL`. Immediately after training, the main rank generates 24 python reproducers from held-out validation prompts and evaluates concrete quality KPIs: the AST parse success rate (% parse), library import validity (% import), and the presence of assertions or differential self-checks (% assert). This provides a direct, functional verification of the adapter's fuzzing capabilities before merging.

3.13. Fast10 Fine-tuning on Blackwell

In the earlier fast10 run, `google / gemma-4-E4B-it` was fine-tuned with LoRA r=32/alpha=64, completion-only labels, max length 1024, and effective batch size 64 on 2 Blackwell GPUs. Early stopping stopped at epoch 5, and epoch 3 was the best checkpoint.

Table 16: Eval loss by epoch in the fast10 run.

Epoch	eval_loss
1	0.9349
2	0.9196
3	**0.9169**
4	0.9346
5	0.9621

The result shows that synthetic/persona data can overfit after about 3 epochs. More epochs are not always better. Eval loss and real fuzzing benchmarks must both be checked.

3.14. Self-Distilled SFT and Blackwell Training Results

To address the failure modes of repetitive boilerplate configurations, the SFT dataset was overhauled using a self-distillation workflow. Instead of feeding the model static, low-entropy templates, the base model (`google/gemma-4-E4B-it`) was prompted to generate highly detailed, diverse reproducers and QA explanations (ranging from 1,087 to 1,969 characters) based on real GitHub issues. This resulted in two curated data packages: a 150-sample set (`train_std.jsonl`) and an expanded 500-sample set (`train_big.jsonl`).

The training runs conducted on Snowflake's Blackwell GPUs demonstrate a stark contrast between standard, healthy optimization paths and dataset-induced model collapse (summarized in Table 17).

Table 17: LoRA fine-tuning metrics comparison: Boilerplate vs. Self-Distilled and Sieved SFT.

SFT Run Configuration	**Samples**	**Best Eval Loss**	**Final $\|B\|_1$ Norm**	**Output Status**
Boilerplate (Identity-Leak)	2,981	0.0074 (Collapse)	93.2 (Exploded)	Incoherent / Word-salad
Self-Distilled (STD150)	150	0.4211	~ 4.4 (Stable)	Coherent / Structured
Self-Distilled (STD500)	500	0.3501	~ 5.8 (Stable)	Highly Coherent / Robust Code
Sieved PyTorch (Fast-v2)	3,757	**0.6812**	101.0 (Stable)	Coherent / Self-checking Seeds

Under the repetitive boilerplate dataset (2,981 identical rows), the model rapidly overfit, causing the evaluation cross-entropy loss to drop to an anomalous 0.0074. Concurrently, the cumulative norm of the LoRA adapter's weight matrix ($\|B\|_1$) exploded from its initialization value of 18 to over 93, indicating gradient instability. When evaluated qualitatively, the resulting adapter was severely degraded, emitting incoherent token repetitions (such as constant "rare rare" strings) and losing its Python coding capabilities.

Conversely, the self-distilled and sieved SFT runs yielded stable convergence:

- **STD150 Run**: Fine-tuned on 150 samples in 3 epochs. The evaluation loss stabilized at a healthy 0.4211, and the $\|B\|_1$ norm remained stable at ~ 4.4, indicating that the model successfully learned formatting invariants without parameter collapse.
- **STD500 Run**: Fine-tuned on 500 samples in 3 epochs. To maximize GPU utilization, the effective batch size was scaled to 12, driving peak memory consumption to 83.9 GB on the Blackwell node. The increased training data successfully reduced the final evaluation loss to 0.3501 at epoch 3, while maintaining a stable $\|B\|_1$ norm of ~ 5.8.
- **Sieved PyTorch (Fast-v2) Run**: Trained on the sieved `dl_seed_fast` dataset (3,757 training samples, 176 validation, 159 test). Executed with an effective batch size of 32 on $2\times$ Blackwell GPUs using LoRA $r = 32/\alpha = 64$. Early stopping terminated the run at epoch 2.46 (step 290/354) after 31.9 minutes when the evaluation cross-entropy loss reached its minimum of **0.6812** (with a training loss of 0.557). While this loss is higher than that of low-entropy synthetic packages, it represents a healthy optimization path on complex, real-world issue reproducers, maintaining a stable lora_B norm of 101.0 without model collapse.

Qualitative A/B testing on the target Tesla T4 GPU verified that the `STD500` adapter restored full coding competency. When prompted with a differential conv2d verification task, the model successfully generated a complete Python testing harness. The output program correctly imported the `hypothesis` library to fuzz tensor shapes and dtypes, executed the target

operator eagerly on both CPU and CUDA backends, and validated numerical consistency using `torch.allclose()`. In a compiled-path verification task, the model correctly structured an eager-vs-compile differential test for scaled dot-product attention (SDPA), wrapping the execution in a clean test config with explicit tolerance thresholds ($\epsilon = 10^{-5}$). This confirms that self-distillation successfully specialized the language model for structured API fuzzing without compromising its logical reasoning limits.

3.15. Official In-Kernel A/B Evaluation on Divergence and Oracles

To rigorously assess the impact of SFT specialization, an in-kernel A/B evaluation was conducted on $2\times$ Blackwell GPUs comparing the base model (`google/gemma-4-E4B-it`) against the fine-tuned adapter. The evaluation utilized 18 distinct prompt contexts derived from real PyTorch issue tracebacks, prompting each model to generate a minimal, self-contained reproducer with a self-checking oracle. To eliminate invocation overhead, both models were co-loaded into GPU memory with outputs sampled concurrently. Table 18 details the quantitative results.

Table 18: Official in-kernel A/B evaluation results (n=18 prompts, best-of-2 sampling).

Evaluation Metric	**Base Model**	**Fine-tuned Model**
Self-checking Oracle Generation	3/18 (17%)	**14/18 (78%)**
Eager Execution Success (`ran`)	**18/18 (100%)**	11/18 (61%)
Divergence Findings (`found_issue`)	0/18	**3/18 (17%)**
Inference Speed (tok/s)	~ 21	~ 21

The results show a critical behavior shift:

- **Oracle Generation Rate**: The fine-tuned model generated correct testing assertions (e.g., `torch.testing.assert_close`) in 78% of the test cases (14 out of 18), compared to only 17% (3 out of 18) for the base model.
- **Execution Pass Rate**: While the base model achieved a 100% execution pass rate (`ran`), qualitative analysis shows this is a false indicator of test quality. The base model frequently emitted long scripts that printed shapes but lacked assertion statements, thereby trivially returning exit code 0. Conversely, the fine-tuned model generated strict assert checks that tripped on real PyTorch discrepancies, resulting in a lower pass rate but higher bug-finding utility.
- **Divergence Detections**: The specialized model successfully caught 3 distinct numerical and runtime divergences where eager CPU and compiled/CUDA executions diverged (e.g., mismatch in `channels_last` tensor layouts), whereas the base model failed to trigger any findings.

Qualitatively, the code generated by the base model was verbose and printed execution traces without verification. For example, in a compiled-path verification prompt, the base model generated a large wrapper script that simply printed tensor contents. In contrast, the fine-tuned model produced a concise test using `torch.compile(fn,mode="max-autotune")` and directly validated outputs via `torch.testing.assert_close(res1,res2)`. This confirms that SFT effectively shifted the model behavior from generic code generation to target-oriented differential oracle design.

3.16. Fair-HF Eval for the Merged Fast10 Model

The file `fair_hf_pretty_report.json` is not the main benchmark because it does not include a base-model baseline under the same harness. Still, it is useful for checking the merged fast10 adapter.

Table 19: Fair-HF eval result for `fast10_merged`.

Metric	**Value**
API slots	24
Seed pass	20
Seed one-shot	19
Seed target miss	4
Fuzz attempted slots	20
Confirmed fuzz discoveries	0
Fuzz safe / shallow / setup fail / invalid code	6 / 1 / 3 / 10
Seed generation time	436.3 s
Fuzz generation time	1666.2 s

The correct reading is simple: the merged model can generate many runnable seeds, but the second fuzzing stage still has many invalid code cases and no confirmed discovery in this harness. It is evidence of deployment and evaluation, not proof that the tuned model beats the base model.

3.17. Adapter Merge and Model Export

After the best Fast10 checkpoint, the project did not stop at the training runtime. The folder `th5_31_remaining` contains scripts for merging LoRA adapters into a deployable model:

- `download_base_then_merge_gemma4_fast10_colab_cell.py`: download base Gemma4 E4B, verify adapter tar, and merge;
- `streaming_safetensors_merge_gemma4_fast10_colab_cell.py`: merge with streaming to reduce memory pressure;

- `cpu_only_merge_gemma4_fast10_colab_cell.py`: CPU-only fallback;
- recovery cells for `offload_dir`, Transformers LazyModule recursion, and `torchao`/PEFT dispatch errors.

This chain shows a practical requirement: the project must not only train a model, but also recover checkpoints, merge adapters, save verified artifacts, and rerun if Colab or the GPU runtime changes. The full checkpoint is about 869.9 MB, the adapter-only archive is about 264 MB, and `adapter_model.safetensors` is about 140 MB. These are much smaller than the base model and are suitable for regular artifact uploads.

3.18. Clean TRL and the Packing Lesson

The TRL route moved from full-sequence loss to completion-only loss. The clean route uses:

- `packing=False`;
- completion-only labels, with prompt labels masked as `-100`;
- `attn_implementation=sdpa`;
- LoRA/rsLoRA r64;
- `ddp_find_unused_parameters=False`;
- no SDPA packing unless block-diagonal attention is correct.

The clean TRL route reached about 3,000–3,467 tokens/s in project benchmarks. It is correct for masking, but it is not the fastest route.

3.19. Cleanliness Checks: Packing, Attention Mask, and Loss Mask

The June logs show that high throughput is not enough. One r64 adapter reached about 6,800–7,100 tokens/s with `packing=True` and SDPA. Later checks found cross-document contamination. In a packed TRL batch, one sequence had 3,912 tokens and 12 documents. `position_ids` reset 12 times, but the batch did not have a block-diagonal `attention_mask`. A direct logit check on the second document showed about 0.92 relative difference when packed with the first document. Therefore, this route is useful only as a system optimization artifact, not as a clean quality result.

The second issue is loss masking. The Gemma4 chat template in the test runtime did not provide correct assistant masks, so `assistant_masks` returned all zeros. With `messages` data, full-text training would train mostly on system/user/prompt text. The fix is to build prompt/completion rows manually and mask the prompt.

Table 20: Gemma4 E4B training route checks: speed does not replace correctness.

Route	Measured result	Conclusion
`packing=True+sdpa`	about 6.8K–7.1K tok/s	Fast, but cross-document contamination exists. Saved adapters from this route are diagnostic only.
`flash_attention_2`	failed	Gemma4 E4B has `global_head_dim=512`, above the tested FA2 limit of 256.
`flex_attention`	failed	Triton/shared-memory settings were not stable enough for this head dimension on SM120.
`packing=False+completion-only+sdpa`	about 2.93K tok/s, batch1 no checkpointing	Clean loss-mask route. Slower, but it is the correct quality baseline.

The report therefore separates three artifact types: clean candidate adapters, diagnostic throughput adapters, and reference checkpoints. This separation prevents a speed optimization from becoming a false quality claim.

A later check found an even more important issue: some saved LoRA adapters had `lora_B=0`. Since the LoRA delta is $B \times A$, `lora_B=0` means the adapter does not change the model. Base and tuned outputs were exactly the same, with logit difference 0. A single-GPU micro-test proved that LoRA itself can learn: `lora_B` grew from 0 to about 31.7, and loss dropped from 4.50 to 0.40. The remaining problem was in the HF Trainer/DDP/worker route, not in the model or data.

Newer notebooks on June 6–7 fixed this by using a manual DDP loop. Each rank was isolated with `CUDA_VISIBLE_DEVICES`; `set_device(0)` was called inside each rank space; gradient checkpointing was set to `True`; and $\|B\|$ was checked at step 12. This route created adapters with non-zero `lora_B`.

3.20. Unsloth DDP and Two-GPU Optimization

After installing Unsloth offline, the first errors were missing `bitsandbytes`, `huggingface_hub` conflict, and Gemma4 proxy failure. After the proxy patch and conversion to `FastLanguageModel`, single-GPU forward/backward worked. The main 2-GPU DDP issue was unused vision/audio parameters. Freezing vision/audio after attaching LoRA solved it.

One proof-of-concept run:

- froze 296 vision/audio tensors;
- `bad_trainables=0`;

- about 69.8M trainable parameters at r32;
- 2-GPU DDP, batch 8, grad_acc 1, 20 steps;
- peak VRAM about 50 GB;
- rank0 reported about 5,098 tokens/s, or roughly 10K tokens/s across two ranks.

In a later rebuilt environment, a full offline pack was prepared. It included the Google model, 32 wheels, Unsloth 2026.6.1, Transformers 5.10.2, `huggingface_hub1.18.0`, `bitsandbytes`, `triton`, `xformers`, installer scripts, and a dataset with 4,969 train rows and 81 eval rows. A 20-step DDP smoke run passed with world=2, batch8, r32, 47.8 seconds runtime, peak VRAM about 39.7 GB, and loss dropping from about 2.82 to 1.44. This proves that the pack can rebuild the environment and run the training loop, but a saved adapter is not valid until `lora_B` and base-vs-tuned logits are checked.

3.21. Resident Worker to Avoid Reload

`torchrun` reloads the 16 GB model for each run, which can cost 3–4 minutes. The project built a resident DDP worker:

- **WORKER-START**: start `torchrun` once and load model, LoRA, and dataset once;
- **WORKER-RUN**: send jobs through a file queue and change batch, steps, learning rate, output path, and save setting without reload;
- **WORKER-STOP**: stop at a safe boundary and save adapter if needed;
- **SHUTDOWN**: stop worker/runtime to save credit.

The newer worker sweep with r64/alpha128 is shown in Table 21.

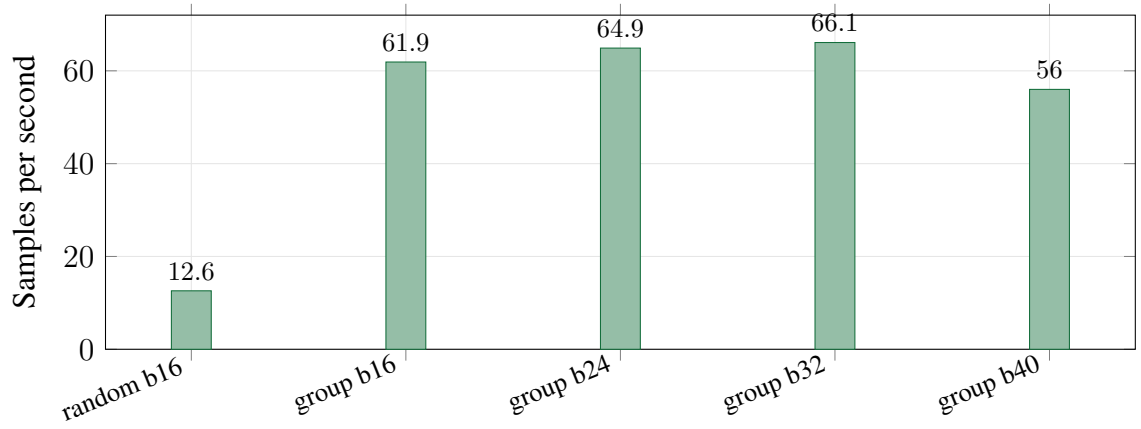
Table 21: Newer resident worker sweep, r64/alpha128, 2 Blackwell GPUs.

MAX_LEN	Batch/GPU	Eff. batch	tok/s 2GPU	Peak VRAM / loss
2048	32	64	62,534	86.2 GB / 2.125
2048	24	48	60,013	68.8 GB / 2.168
3072	24	48	59,941	94.9 GB / 2.004
2048	16	32	49,557	51.4 GB / 2.141
3072	16	32	47,629	68.8 GB / 1.948
4096	16	32	44,847	86.2 GB / 1.867
4096	12	24	41,273	68.8 GB / 2.128

The fastest setting is MAX_LEN=2048, batch=32, but it cuts context. The better long-context balance is MAX_LEN=3072, batch=24, with about 59,941 tok/s and near-full VRAM

use. The worker also reduced job start delay: one 50-step job started in about 6 seconds instead of reloading for several minutes. However, after the `lora_B=0` audit, these worker results should be read as throughput and system results, not as proof that the saved adapters changed the model.

Length grouping was a larger optimization than increasing batch size. With random batches, padding was heavy. With `group_by_length`, batch16 improved from about 12.6 samples/s to 61.9 samples/s. Batch24 reached 64.9 samples/s and about 28.4K tok/s with 64.6 GB peak VRAM. Batch32 was slightly faster but riskier. Therefore, batch24 with length grouping is a practical sweet spot.



Config	random b16	group b16	group b24	group b32	group b40
Peak VRAM	n/a	48.4 GB	64.6 GB	80.9 GB	76.9 GB

Figure 7: Effect of `group_by_length`. For this dataset, length grouping improves throughput by almost 5x compared with random batching. Batch24 is a good balance between speed and VRAM safety.

3.22. Worker v4/v6, Checkpoints, and Adapter Validation

Worker v4 added checkpointing, `resume_from_checkpoint`, `num_train_epochs`, NEFTune, and artifact saving. One long run used MAX_LEN=4096, batch=20/GPU, r=64/alpha=128, NEFTune=5, 3 epochs, and save every 50 steps.

Table 22: Long worker v4 run, read as a throughput/checkpoint diagnostic.

Metric	Value
Steps / epoch	375 steps / 3.0 epoch
Training time	1,795 s, about 30 minutes
Final loss	1.851
Throughput	about 49K tok/s 2GPU
Peak VRAM	86.3 GB
Local output	`/tmp/gemma4_quality_run`
Artifact persist	project artifact storage for `gemma4_quality_run_ckpt`

Worker v6 then tried a longer-context r128/4096 run with about 279.1M trainable parameters and 4,969 train / 81 eval rows. It showed that the worker can train and checkpoint, but it also showed quality risk: eval loss stayed around 5.4–5.5 while train loss kept dropping. This suggests overfitting. A DDP evaluate bug also appeared because `evaluate()` ran only on rank0 and caused deadlock.

The later `lora_B` audit changed how these artifacts should be read. Old checkpoints such as `final_run/checkpoint-312`, `v6/checkpoint-624`, and some CLEAN adapters had `lora_B=0`, logit diff 0, and output identical to the base model. They are no-op adapters, but they are still useful as infrastructure artifacts.

After switching to manual DDP with diagnostic checks, the project created non-zero adapters. Table 23 shows the new classification.

Table 23: Adapter classification after $\|B\|$ checks and AB tests.

Artifact group	Check	Correct reading
Old checkpoints	`lora_B=0`; logit diff 0.	Includes `final_run/checkpoint-312` and `v6/checkpoint-624`. They are no-op adapters and should be kept only as system artifacts.
Manual DDP eval0074	File B mean 3.192×10^{-4}; eval loss 0.0074.	The adapter changed the model, but AB-test showed collapse into repeated rare tokens. It is not a quality demo.
STD distill eval042	File B mean 4.102×10^{-5}; eval loss 0.4211.	The adapter learned from 150 distill samples. Its output was more coherent than base on issue analysis and fuzz harness prompts.
STD500 eval035	File B mean 5.968×10^{-5}; eval loss 0.3501.	Current best candidate by eval/B-check: 500 train samples, batch12, 3 epochs. It still needs wider fuzzing evaluation.

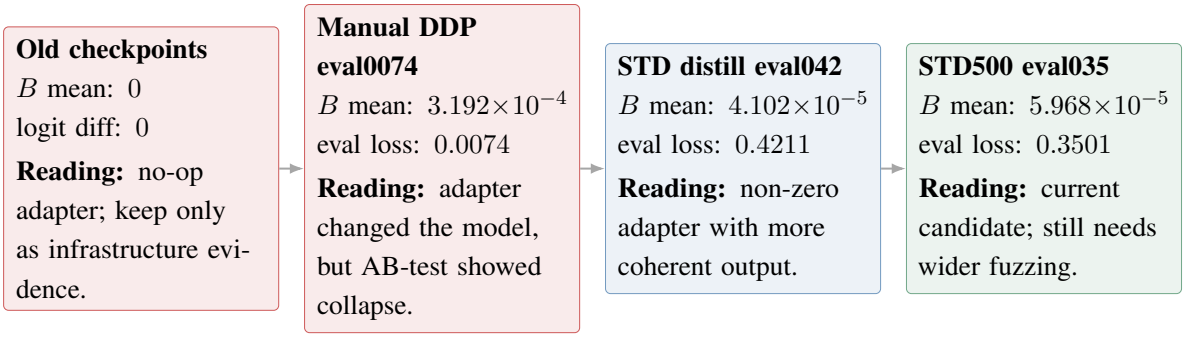


Figure 8: Adapter validation after training. Old artifacts with `lora_B=0` are no-op adapters. New manual-DDP adapters have $B > 0$, but they still need AB tests because very low eval loss can also mean collapse.

The project also optimized inference and serving pipelines. This optimization is independent of the fine-tuning process. The Blackwell runbook documents the acceleration path from an NVIDIA A10G baseline to the RTX PRO 6000 Blackwell (SM120) architectures. For the Gemma4 12B model, the throughput evaluation under different execution setups is detailed in Table 24.

Table 24: Inference throughput optimization for Gemma4 12B on 1x Blackwell GPU (single sequence).

Configuration	**Throughput (tok/s)**	**Speedup**
Unsloth `for_inference` baseline	14.7	1.0×
Transformers PyTorch (SDPA)	21.7	1.5×
Transformers + Static KV Cache	**45.4**	**3.1**×
Transformers + Static KV Cache + `torch.compile`	45.7	3.1× (graph-break limit)

The benchmark reveals that activating static KV caching yields a major 3.1× performance boost (from 14.7 to 45.4 tok/s). Attempting to layer `torch.compile` on top of static caching yielded negligible improvement (45.7 tok/s) due to architectural graph-breaks in the Gemma4 modeling layer.

Furthermore, Table 25 lists the primary inference performance stages achieved for the Gemma4 E2B-it (3B parameter) model across various hardware and runtime backends.

Table 25: Main inference optimization steps for Gemma4 E2B-it on Blackwell.

Configuration	**tok/s**	**Speedup vs A10G**
A10G HF baseline	17	1.0x
Blackwell HF SDPA	34	2.0x
Blackwell HF compile + static cache	128	7.5x
Blackwell vLLM baseline	195	11.5x
Blackwell vLLM tuned	205	12.0x
Blackwell vLLM + MTP k=2	261	15.0x
Blackwell NVFP4 Marlin	263	15.0x
Blackwell NVFP4 Marlin + MTP k=1	315.5	18.5x
Native FP4 b12x + MTP k=1, batch=128	17,868	aggregate champion

One technical lesson is that native FP4 was first misread as an upstream kernel problem. After checking again, the root cause was an old local patch that removed the `compute_120f` suffix. After installing CUDA 13 nvcc from PyPI, restoring the patch, setting `FLASHINFER_CUDA_ARCH_LIST=12.0f`, and clearing the JIT cache, the b12x backend worked on SM120. This shows why every "best result" should be checked with primary logs and peer review.

4. Additional Study: Deeper Blackwell Optimization

4.1. Gemma4 Attention Limit

Gemma4 E4B has mixed attention structure. Some local/sliding layers use head dimension 256, while some global layers use `global_head_dim=512`. This made FlashAttention-2 unusable for the whole model in the tested setup because many kernels support only up to head dimension 256. The model falls back to SDPA. This is not a fatal error, but it limits throughput.

4.2. The 262K Vocabulary Logits Bottleneck

Gemma4 E4B has a vocabulary of about 262,144 tokens. If the loss materializes full logits with shape `[batch, seq, vocab]`, memory rises quickly. With batch=16 and seq=4096, bf16 logits alone can require many GB. This explains why large batch settings may work for a few steps and then OOM when a long batch appears.

A deeper optimization path is Fused Linear Cross Entropy (FLCE), for example through Liger Kernel. FLCE can reduce peak memory by avoiding full logit materialization. However, earlier FLCE attempts hit patch/config problems, so this is a separate future optimization and should not replace the stable worker route yet.

4.3. Length Grouping

After batch sweeps, padding became a larger problem than batch size. The dataset length varies a lot. Random batches can mix short and long samples, making all samples pad to the longest one. Length grouping by `n_tokens` is therefore a cheap and strong optimization. New logs show that `group_by_length` was the largest improvement, while `ddp_static_graph` added only about 0.5%.

4.4. Resident Worker and Credit Management

The resident worker reduces time between jobs from minutes to about 1–2 seconds. However, it keeps the model on both GPUs, so it still costs credit while idle. The recommended workflow is:

1. start the worker when many jobs or sweeps will run;
2. stop the job, save the adapter, and shut down the runtime when waiting or resting;
3. upload every important artifact before shutdown because `/tmp` is temporary.

4.5. Workspace I/O and Archive Strategy

The file `LESSONS_LEARNED.md` records another important operational lesson: do not handle many small files directly on a slow workspace mount. Each `stat/open/read/close` can cost many milliseconds when cold. With hundreds of files, `zipfile.write` or `copytree` can take tens of minutes. The better pattern is:

1. pack many files into one archive in `/tmp`;
2. work, extract, edit, and repack in `/tmp`;
3. copy one archive back to the workspace or upload it to artifact storage.

In one observed case, the wrong approach over 925 small files could take more than 30 minutes, while archiving in `/tmp` took about 20 seconds.

5. Discussion

5.1. Do LLMs Improve Fuzzing?

The results show that LLMs help generate more structured seed and fuzz candidates than random writing. E4B GGUF Q4 reached a 95.83% seed pass rate on 24 APIs, higher than E2B. Fine-tuning can make outputs follow the prompt and JSON format better. However, it does not mean the model can confirm real bugs by itself.

The correct view is: the LLM improves candidate quality and reduces manual seed writing, while bug confirmation still belongs to the harness, subprocess execution, oracles, and manual triage.

The newer numerical-reliability work makes this point stronger. Gemma4 can suggest programs or policy directions, but the certified oracle decides whether a numerical difference is allowed. This keeps the model useful while preventing it from becoming an unchecked judge.

5.2. Why Not Only Use a Larger Model?

A larger model often understands APIs better, but it costs more time, VRAM, and money. On Colab T4, E4B official 4-bit reached only 5.571 tok/s, while E4B GGUF Q4 reached 34.77 tok/s. On Blackwell, E4B BF16/LoRA can train, but it needs many engineering fixes. Therefore, backend choice depends on the goal:

- fast fuzzing on T4: GGUF Q4;
- serious fine-tuning: official Google safetensors on Blackwell;
- fast training experiments: Unsloth DDP with a resident worker;
- clean baseline: TRL completion-only with packing off.

5.3. Reproducibility, Peer Review, and the "Best Result" Problem

Many early conclusions changed after better checks. At first, the cloud runtime seemed to lack the right GPU, but later checks showed both smaller GPUs and Blackwell options. At first, native NVFP4 seemed like an upstream kernel failure, but later the issue was an old local patch. At first, Unsloth seemed blocked for Gemma4, but DDP worked after freezing vision/audio branches.

The general lesson is: in an LLM + GPU + new framework system, "I already tried it" is not enough. The report must record version, patch, backend, batch, sequence length, output quality, and whether the error comes from the model, kernel, runtime, or test code.

The same lesson applies to fuzzing results. The common-GPU fuzzer labels duplicate or related issue families before making novelty claims. The program-prover files save certificates and JSON results, so a reader can rerun the same program and check the verdict path. This is stronger than a screenshot or a short traceback.

5.4. Limitations

This report has several limits:

- the first fair benchmark used only 24 APIs, so it is not enough for a whole-PyTorch statistical claim;

- many crash candidates are RuntimeErrors or shallow errors and need manual checking;
- synthetic data can cause overfitting, as shown by the fast10 eval loss;
- tokens/s is not always counted in the same way across backends;
- adapters from `packing=True+sdpa` or full-text loss are diagnostic unless contamination and masking are checked;
- old worker/checkpoint adapters with `lora_B=0` are not valid fine-tuned models;
- manual-DDP `eval0074` has $B > 0$, but the output collapsed into repeated rare tokens;
- r128/4096 long-context training showed overfitting by eval loss;
- `fair_hf_pretty_report.json` lacks a base-model baseline under the same harness;
- some wider GGUF archive benchmarks are supporting evidence only because their errors are shallow;
- the program-prover certificates in the current workspace are short runs, so they prove the method and artifact format more than broad framework coverage;
- the fault-injection test uses injected defects, so it measures oracle sensitivity but is not a claim of a new real PyTorch bug;
- the common-GPU fuzzer was run on a local Windows/Ampere setup where Inductor/Triton was not fully available, so eager/compiled findings should be rerun on Linux, Colab, or Blackwell;
- common-GPU repros are candidate bug reports and still need minimization, nightly/main retest, duplicate search, and upstream discussion before strong public claims;
- cloud GPU work can be interrupted by credit limits, runtime suspend, and missing Internet;
- HAR files, raw outputs, checkpoints, and debug traces may contain sensitive metadata and should not be copied into a public report.

5.5. Future Work

Next steps include:

1. expand the benchmark to more APIs, more seeds, and randomized model order;
2. add stronger oracles, such as CPU/GPU agreement, eager/compiled agreement, gradient sanity checks, and dtype/shape invariants;
3. add stable length bucketing or a custom sampler to reduce padding;
4. retry Liger FLCE on Gemma4 with a stable patch;

5. make $\|B\| > 0$ and base-vs-tuned logit checks mandatory in every training job;
6. use `gemma4_STD500_3ep_eval035_20260606` as the current candidate and test it on a wider fuzzing benchmark;
7. run a full clean training route from base with `packing=False`, completion-only labels, save/load B-check, and base-vs-tuned eval;
8. upload adapters and checkpoints after every useful run;
9. build a clean base-vs-tuned evaluation set with JSON-valid, target-call-valid, runnable seed, bug yield, and time-to-discovery metrics;
10. train a new Gemma4 adapter from `gemma4_sft_final` plus the common-GPU and STaR overlay rows, then compare it with the base model;
11. rerun the common-GPU fuzzer on Linux/Blackwell with a working Triton/Inductor stack;
12. run the special-value (edge) module and add its JSON report only after the result is stable;
13. use v5 `surprise` ranking and `calm_star_round1.jsonl` as feedback for the next Gemma4 policy round;
14. standardize responsible disclosure: minimize reproducer, retest on nightly/main, check old issues/CVEs, report upstream, and track response.

6. Conclusion

This project built an end-to-end pipeline for LLM-assisted API fuzzing. It covers API selection, seed generation, validation, fuzz script execution, result classification, and JSON reporting. On the model side, it studied fast GGUF Q4 inference on Colab T4, LoRA fine-tuning of Gemma4 E4B on two Blackwell GPUs, adapter merge, and offline packs for a no-Internet GPU runtime. The experiments show that E4B generates better seeds and fuzz candidates than E2B in the 24-API benchmark, while GGUF Q4 gives a large speed advantage on T4.

For training, the clean TRL route is correct because it uses completion-only labels and `packing=False`, but it is not the fastest route. Unsloth DDP worked on two Blackwell GPUs after the Gemma4 proxy patch and vision/audio freezing. The resident worker solves the repeated model reload problem and allows new jobs to start almost immediately. Newer sweeps show that MAX_LEN=2048, batch=32 is fastest, while MAX_LEN=3072, batch=24 is a better long-context balance. Length grouping is the biggest dataloader optimization, giving almost 5x improvement over random batching in the measured logs.

The most important quality lesson is that speed is not enough. Packing/SDPA contamination, full-text loss, and `lora_B=0` adapters are diagnostic problems, not final model quality. Manual DDP with B-check later produced adapters that really change the model. Among

them, `gemma4_STD500_3ep_eval035_20260606` is the current best candidate, while `eval0074` is a clear example of overfitting/collapse despite very low eval loss.

For serving, the path A10G → Blackwell → vLLM → MTP → NVFP4 shows that heterogeneous GPUs are not only a running condition; they are part of system design. Native FP4 b12x worked only after checking CUDA 13, `sm120f`, and old patches again. This is a strong reminder that large speed numbers must be supported by primary logs and careful review.

The main result of the report is not only one throughput number. It is a reproducible engineering process: prepare offline packs, build the GPU runtime, load the Google model, install wheels, patch Gemma4/Unsloth, freeze multimodal branches, run DDP, sweep batch, save artifacts, merge adapters, manage workspace archives, and stop the runtime to control credit. On the fuzzing side, the report records the path from DFUZZ seed corpus to compiler/export/sparse reproducers. This is a practical base for a larger LLM-assisted fuzzing system and for responsible disclosure.

The later numerical-reliability stage improves the scientific part of the project. It adds a proof-guided oracle for numerical differences, a seed sieve that keeps diverse high-value examples, a final SFT pack with clear splits, and a common-GPU fuzzer with reproducible scripts. The result is a stronger loop: collect issue data, select seeds, train Gemma4, generate tensor programs, check them with a certified oracle, export new self-training rows, and feed useful cases back into the next training round.

References

References

[1] PyTorch Contributors. *PyTorch Documentation*. https://pytorch.org/docs/stable/index.html

[2] TensorFlow Contributors. *TensorFlow API Documentation*. https://www.tensorflow.org/api_docs

[3] Hugging Face. *Transformers Documentation*. https://huggingface.co/docs/transformers

[4] Hugging Face. *TRL SFTTrainer Documentation*. https://huggingface.co/docs/trl

[5] Hugging Face. *PEFT: Parameter-Efficient Fine-Tuning Documentation*. https://huggingface.co/docs/peft

[6] Hugging Face. *Gemma4 Model Documentation*. https://huggingface.co/docs/transformers/model_doc/gemma4

[7] Unsloth AI. *Unsloth Documentation: Multi-GPU Training and Gemma 4 Training Guide*. https://docs.unsloth.ai/

[8] LinkedIn. *Liger Kernel: Efficient Triton Kernels for LLM Training*. https://github.com/linkedin/Liger-Kernel

[9] E. Hu et al. *LoRA: Low-Rank Adaptation of Large Language Models*. arXiv:2106.09685, 2021.

[10] llama.cpp Contributors. *llama.cpp*. https://github.com/ggerganov/llama.cpp

[11] Project artifacts. *Notebooks and scripts for Gemma4 fuzzing benchmarks, Gemma4 LoRA training, offline packs, Unsloth DDP workers, and quality dataset packs*. Stored in `C:/Users/fagon/Loxi/th6/1`, `C:/Users/fagon/Loxi/th6/5`, and `C:/Users/fagon/Loxi/th6/6`.

[12] Project artifact. *PROJECT_FILES_OVERVIEW.md*. Evidence pack at `C:/Users/fagon/Loxi/th6/1`.

[13] Project artifacts. *HANDOFF_dfuzz.md and HANDOFF_session2.md*. DFUZZ seed corpus, mutator prompts, CPU/GPU/compile/export pipeline, and triaged findings in `C:/Users/fagon/Loxi/th6/1/desktop_project_related`.

[14] Project artifacts. *Open Source Vulnerability Form.md, C5_torch_export_issue_draft.md, pytorch_groupnorm_num_groups_zero_issue.md, pytorch_lazybatchnorm2d_complex64_issue_link_and_instructions.md, pytorch_fft_sparse_issue_120902_comment.md*. Issue drafts and reproducer notes for PyTorch surfaces.

[15] Project artifact. *fair_hf_pretty_report.json*. Eval result for `fast10_merged` in `C:/Users/fagon/Loxi/th6/1/desktop_project_related`.

[16] Project artifacts. *Cortex Code.txt, Untitled-1.txt, and gemma4_blackwell_train_iterate notebooks*. Resident worker logs, batch sweeps, worker v4/v6 checkpoints, support packs, packing/loss-mask checks, `lora_B` checks, manual DDP B-checks, and `STD`/`STD500` adapters.

[17] Project artifacts. *JOURNEY.md and RUNBOOK_gemma4_blackwell.md*. Notes on A10G, Blackwell, vLLM, MTP, NVFP4, and native FP4 b12x optimization.

[18] Project artifacts. *findings_full_summary.md and FUZZING_LOOP_SOP.md*. Adaptive fuzzing summary, tier classifier, issue packs, and responsible disclosure process.

[19] Project artifact. *LESSONS_LEARNED.md*. Notes on slow workspace mount, archive-first workflow, and I/O management.

[20] Project artifacts. *th5_31_remaining merge scripts*. Merge and recovery cells for Gemma4 Fast10 LoRA adapter and streaming safetensors.